

34th CIRP Design Conference

An Investigation on Utilizing Large Language Model for Industrial Computer-Aided Design Automation

Haoxuan Deng^{a,*}, Samir Khan^a, John Ahmet Erkoyuncu^a^a*School of Aerospace, Transport and Manufacturing, Cranfield University** Corresponding author. Tel.: +44 (0)1234 750111. E-mail address: haoxuan.deng@cranfield.ac.uk**Abstract**

Text-to-image or 3D content generation has attracted attention in academia and industry because of its impressive performance in high-fidelity content generation with natural language commands. However, its widespread application in industrial computation-aided design (CAD) is not yet found. The main issue lies in the end-to-end generation through implicit probabilistic learning from datasets. This presents difficulties in precise, controllable, and transparent creations. Also, it is time-consuming and labor-intensive for designers to verify and customize the generated result to fit their demands. Furthermore, popular generative models offer limited support for early-stage engineering computations, leaving tedious and massive calculations to human engineers. To overcome these challenges, this paper proposes a method to apply large language model (LLM) to assist standard CAD design workflow on both engineering computation and precise 3D geometry generation. During the designing process, LLM can support the computation process with human guidance and generate accurate 3D shapes with the capability of sequential code representation and transformation. To verify the feasibility of the proposed method, a use case for a one-stage reduction gear system consisting of two gears will be given as a proof-of-concept demonstration. The result shows that the LLM can carry out the computation and get the result as expected with only a few lines of human guidance and feedback. Then, it can create CAD models with accurate parameters aligned to the computational results. It can cope with various requirements given by users and allow users to further modify the generation using the same CAD environment without much effort. Authors believe that this piece of work can shed light on utilizing the capability of LLMs for a certain part of task automation based on human beings' intentions. With the rise of LLMs and relevant technologies, conventional CAD may shift to a more user-friendly AI-aided design (AIAD) diagram.

© 2024 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 34th CIRP Design Conference

Keywords: Design automation; Large language model; Compute-aided design**1. Introduction**

Over the past five years, dramatic progress could be found in the domain of artificial intelligence (AI), including computer vision, sound recognition, and video processing. In particular, the large language model (LLM) is one of the most well-known developments because of its impressive power in natural language processing. With billions of parameters pre-trained on extensive textural corpus, LLM emerged with a surprising reasoning capability that can allow the model to analyze and tackle some tasks by itself without much human intervention. In addition, LLM can detect intentions from natural language

instruction, or say prompt, given by human users. Therefore, research on using LLM in context-aware workflow automation based on simple commanding is becoming popular in the AI community.

It is thus appealing to investigate applying LLM in the context of manufacturing because there are large numbers of knowledge and labor-intensive processes that call for automation to cut down human workload. For instance, conventional mechanical design is to construct a set of 2D or 3D shapes to satisfy both geometry and performance criteria for engineering applications. Most of these design tasks in industry usually begin with requirements documentation, engineers are

required to analyze the documentation line-by-line and carry out calculations to obtain necessary parameters for each part of the system. After that, CAD packages will be used for 2D sketching and 3D solid construction. During this pipeline, requirements file analysis, design computation, and 3D modeling can be conducted by LLM with its capabilities in natural language understanding, logic reasoning, and text-to-image generation. Thus, it is worth investigating the automation of the CAD workflow by chaining these sessions together under the power of LLM.

Based on this idea, this paper attempts to explore how to integrate LLM in the design process to offload some human workloads in this process. To achieve this target, the main issue is *how to enable the LLM to establish a perceive-reasoning-action chain for planning and taking actions based on the interaction with human users*. To better demonstrate the proposed method, use cases for a gear system design based on given requirements will be shown to reveal the process and verify the feasibility. The contributions of this paper can be summarized into twofold:

- Propose a framework for chaining design requirement analysis, engineering computation, and model creation with LLM for CAD workflow automation.
- Verify a method for precise and editable text-to-CAD model generation with multiple components.

The rest of the paper will be organized into the following structure. The second section will list some relevant literature that relates to this work. After that, the framework of the proposed methodology will be illustrated. In the next section, a use case for a gear system design will be presented with experimental configuration, results, and discussion. The next sections will provide some analysis of future works based on the discussion of the use case. Finally, the paper will end with a brief conclusion. In this paper, the authors would like to provide insight into the future direction of human-AI collaboration for mechanical design in manufacturing.

2. Related Work

2.1. Natural language guided images and 3D content generation

Using textual instruction to guide the generation of images or 3D content is not a new concept and the first development can be traced back to the proposal of the AlignDRAW in 2015 [1], and many researchers kept on contributing to this topic since that time. The most well-known state-of-the-art approach is DALL-E2 [2] text-to-image generator developed by OpenAI. There are three main components in DALL-E2, namely text encoder, text embedding to image embedding converter, and image decoder. Text encoder is a model for transforming a piece of textual description into a high-dimensional vector, or say text embedding, which is a format that computers can understand and operate efficiently. Later, the text embedding will be fed into another trained model to map to an image embedding (similarly, a high-dimensional vector for an image representation). In detail, this model is pre-trained on millions of annotated text-image pairs, thus it can establish a mapping between a piece of text and an image pattern. Finally, an image

decoder, commonly a diffusion model [3], will be applied to decode the image embedding back to an image. Another method is to decompose an image as a combination of various code entries, just like discretizing an image into several puzzle pieces. Then, apply a sequence-to-sequence model like Transformer [4] to translate a sequence of tokens in a prompt to such code entries to synthesize a highly realistic image. A representative instance of this branch is Parti developed by Google [5].

The same breakthrough can also be found in the context of 3D content generation based on human natural language. From a high-level perspective, a popular approach is to reconstruct a 3D shape from an image generated by text-to-image technology, such as DreamFusion [6] and Magic3D [7].

2.2. Prompts and large language model autonomous agents

Large language models (LLMs) have proven their powerful capabilities in multiple task performance such as language translation, natural language understanding, sentiment analysis, text summarization, and generation. However, also as a kind of generative model, they suffer from probabilistic uncertainty and cause a severe *Hallucination* problem. As a result, they will produce incorrect and unrealistic information in their responses to human beings. Hence, LLMs will cause misleading and confusion to their users, posing a challenge to reliability and trustworthiness. Furthermore, LLMs with billions of parameters are trained on a large amount of unpublic datasets, it is challenging to allow people to understand, interpret, and validate those conclusions drawn by LLMs, which is referred to as the explainability issue of deep learning. For these reasons, there is still a long way to go to enable LLMs to penetrate scenarios involving accurate operations or high risk.

Prompting engineering, meaning carefully crafting an instruction to command LLMs to get satisfactory responses, is an effective way to alleviate the hallucination issue. Instead of trial-and-error testing prompts for LLMs to conclude immediately (zero-shot prompting), Wei et al. [9] found that guiding the LLMs using chain-of-thought prompting to decompose a problem into multiple sub-tasks and gather answers to each small problem can lead to an increase in success rate of complicated task performance, such as math reasoning and arithmetic operation. On top of that, Yao et al. [10] developed ReAct prompting method to integrate action by using external tools and obtained observation from tool calling into LLMs' reasoning traces. Experiments suggested LLMs can boost their performance on interactive text gaming and online shopping tasks compared to conventional imitation learning or reinforcement learning methods.

Based on this advantage, a new concept called *LLMs Agent*, or *AI Agent*, is proposed to enable more practical applications of LLMs in different domains. The core idea is that LLMs can be treated as a controller to generate a sequence of actionable steps based on their reasoning with a set of external toolkits such as databases, calculator, and web browsers. In each step, LLMs can produce more concrete intermedium outcomes by integrating results from previous steps and tool calling, which will then be gathered to produce the result. In this way, the

hallucination of LLMs can be alleviated because the outcomes from tool calling are more reliable than a pure generation. Additionally, people can understand and track the logic of the problem-solving given by LLMs, which can be reviewed and refined by human users through the natural language interface. Fruitful early-stage demonstrations of the LLMs agent in various domains can be found in [11]. Moreover, some research [12], [13] also shows that LLMs agents with prompt engineering can conduct cross-modality tasks like image understanding, reasoning, and editing using textual instructions, without training. This indicates a new way for controllable text-to-image or 3D content generation.

2.3. Gaps and challenges

Most generative models are dependent on implicit representations (e.g., high-dimensional vectors) of the shape and patterns, which are difficult to visualize, understand, and operate by human beings. In fact, what generative models do is to approximate a probabilistic distribution over a given dataset as closely as possible and to generate new items by sampling on the learned distribution. Because of the inherent randomness, it is challenging to promise and reproduce accurate operations during 3D modeling, even though some research like ControlNet can improve the controllability of generation outputs [8]. Therefore, they are yet mature to be applied in parametric design for CAD purposes.

Prompting engineering and LLMs agents are promising for solving this problem and show potential in participating in other sessions in CAD workflow based on powerful natural language processing ability. However, how these techniques can be applied in CAD is yet fully explored, and investigations of the impacts and benefits of bringing relevant ideas into this domain are nearly silent.

3. Illustration and implementation of the design workflow with the large language model

3.1. Initial analysis

As introduced in Section 2, a way for text-to-3D geometry generation, without using pre-trained text-to-image or text-to-3D content models, is the application of prompting engineering and external tool usage. An initial idea is to prompt LLMs to generate *formal language representations of shapes*, such as L-system [14] and shape grammar [15] for describing a geometric pattern of a product. Then visualize the 3D geometry based on generated formal language with rendering tools like Turtle Graphic Library or Unity. In this way, it is feasible to obtain controllable designs by generating a specific design language with LLMs as shown in the bottom branch of the workflow with the red cross mark in Figure 1. Nevertheless, the problem is the difficulty of setting constraints to restrict the topology of the generative shape by textual strings. Also, the generative process involves iterative and parallel operations, which is difficult to follow for human beings. Thus, designers may struggle to set up proper textual rules at the beginning of the design. Automating a rule-based system configuration heavily depends on human intuition and experience, and it is still an

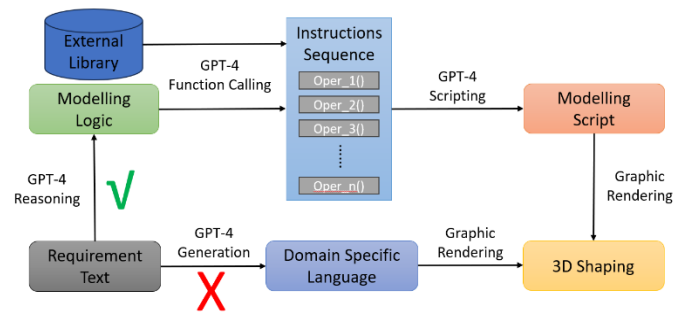


Figure 1. Overview and comparison of workflows for using LLM (GPT-4) in 3D shape construction.

opening question in LLMs research. Additionally, the generated code quality cannot be promised without mistakes because of the hallucination characteristics of LLMs that may cause unexpected results even with clear descriptions of rules and operations.

An alternative option to create a precise and editable 3D shape is shown in the top branch with the green tick icon in Figure 1. The essential idea is to abstract the modeling process by considering how to model. LLMs can reason on requirement documents to produce a logic on how to solve a problem. Based on the reasoning result, LLMs can automatically decompose the logic in a sequence of atomic operations (instructions sequence), and link to functions defined in external standard libraries, to generate a code snippet for shape creation. In this way, LLMs do not directly generate a code binding to a shape, instead, they compile a script consisting of clearly predefined functions according to reasoning logic to achieve a more concrete and trackable generation. In short, *the main idea is to avoid a direct generation of creation but to generate the logic of creation by LLMs*.

A similar idea can be found in the work of 3D-GPT [16], where a multi-agent approach is applied to allow an LLM, GPT-4 [17] in this case, to first decompose the user's instruction into a bunch of clear sub-task descriptions. Based on these descriptions, GPT-4 will select the most relevant functions from a standard library in Blender to code a Python script. Later, the generated script can be used to operate 3D objects by sequential function callings to finally produce a 3D model. In this paper, a similar idea will be applied but without using the multi-agent approach for a basic proof-of-concept implementation. The focus on generations in this work is the precise generation of multiple shapes that can be checked and modified for customization. All other techniques can be investigated for future exploration.

3.2. Overview of the proposed method

The overview of the proposed idea is illustrated in Figure 2. In this paper, GPT-4 will be used to conduct three sub-tasks namely parametric computation, instructions sequence construction, and coding model scripting. In the first step, an initial prompt including the user's design requirements will be fed into GPT-4. Then GPT-4 will carry out the parametric computation and return design documentation to people. This step can iteratively run until engineers are satisfied with the computational parameters after reviewing the documentation.

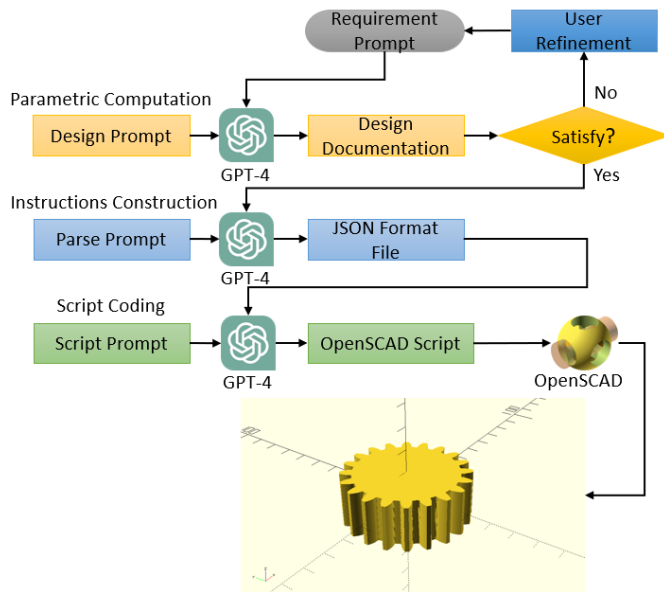


Figure 2. The overview of the proposed method.

After that, the results in design documentation will be structured into several textual instructions with parameters as shown below.

Gear 1: driven gear, module(), teeth number(), height(), ...
Gear 2: driving gear, module(), teeth number(), height(), ...

In the next step, GPT-4 will parse a JSON file to formalize the instructions and tokenize the natural language instructions into a list of keywords and numerical values, so that GPT-4 can further focus on essential parameters. Based on the JSON file, GPT-4 can select the most relevant function from a given library and fill the function with parameters to write the script for 3D modeling. The script will then be rendered by a CAD graphic package to generate the 3D model.

3.3. Implementation

In this work, GPT-4 will be used via the official API published by OpenAI [17]. To better utilize the reasoning ability of the GPT-4, the chain-of-thoughts prompting technique will be applied, and it is simple to implement by just adding an extra instruction, saying “Please analyze the process step by step” in the prompt. It can enable the LLMs to derive their conclusions by multiple analysis, rather than jump into the answer directly so that the reasoning result can be more reliable.

In each step during the workflow, there will be a prompt to guide the LLMs to follow some working patterns or given examples. For the interactive computation process, the standard of the documentation writing, the format of the result, and some restrictions on parameter selection (e.g., standard gear module number) will be provided. In the instruction construction and formalization session, the prompt will define the JSON file that GPT-4 should follow. Finally, the scripting prompt will include the coding procedure to let GPT-4 write the OpenSCAD script.

The graphic rendering toolkit in this case is OpenSCAD [18], a fully open-source CAD package using formal programming language for 3D modeling. Unlike other well-known CAD software such as SolidWorks or Catia, it is not an interactive

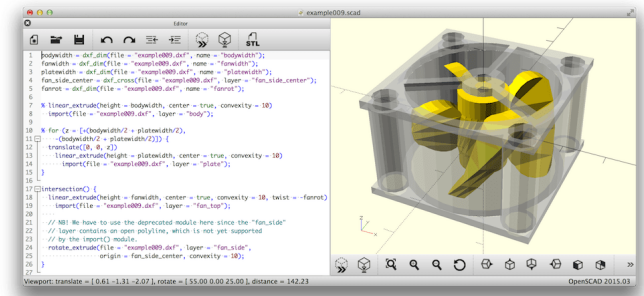


Figure 3. A screenshot of the OpenSCAD interface for 3D modelling (from <https://openscad.org/> [18])

modeling package via a graphic user interface. Instead, it is a compiler that requires programming a script to construct 3D objects. Figure 3 shows a snapshot of building a fan assembly model with script coding in the text board on the left and the rendering result in the interactive window on the right. In this way, the script generated by GPT-4 can be processed by OpenSCAD to get the CAD model.

According to the workflow in Figure 1, GPT-4 is not going to generate the OpenSCAD script directly, rather, it will use atomic functions defined in its library like *translate* (x, y, z) and *rotate* (α, β, γ) to operate objects parametrically. For gear profile and shape construction, an external library [19], also written in OpenSCAD scripting language, will be used for function calling by GPT-4.

4. Use case: one-stage reduction gear system design with CAD model generation

4.1. Experimental setting up

For a proof-of-concept implementation, in this paper, a one-stage reduction gear system will be used to demonstrate how the design process can be automated and evaluated by human feedback, and how the satisfactory results can be used to generate 3D models according to obtained parameters.

In this paper, the axis of two gears is parallel and the desired gear ratio and the distance between two gear axes will be given as design requirements. Other design parameters such as gear height and pressure angle will also be given for simplification. This information is gathered and represented as a textual description to be fed to GPT-4. The expected outputs include design documentation of the computational process and the final design results, a JSON file of design parameters, and an OpenSCAD script for model rendering. The obtained script will be rendered by OpenSCAD to see whether the 3D shape can be generated properly as expected.

4.2. Results

During the process, the design documentation, JSON file, and script can be compiled successfully, which can all be reviewed and modified by human users. The generation of 3D shapes is shown in Figure 4 with different initial design requirements. The text in the red box is the requirement description as input, while textual instructions in the blue box contain obtained parameters computed by GPT-4. The image

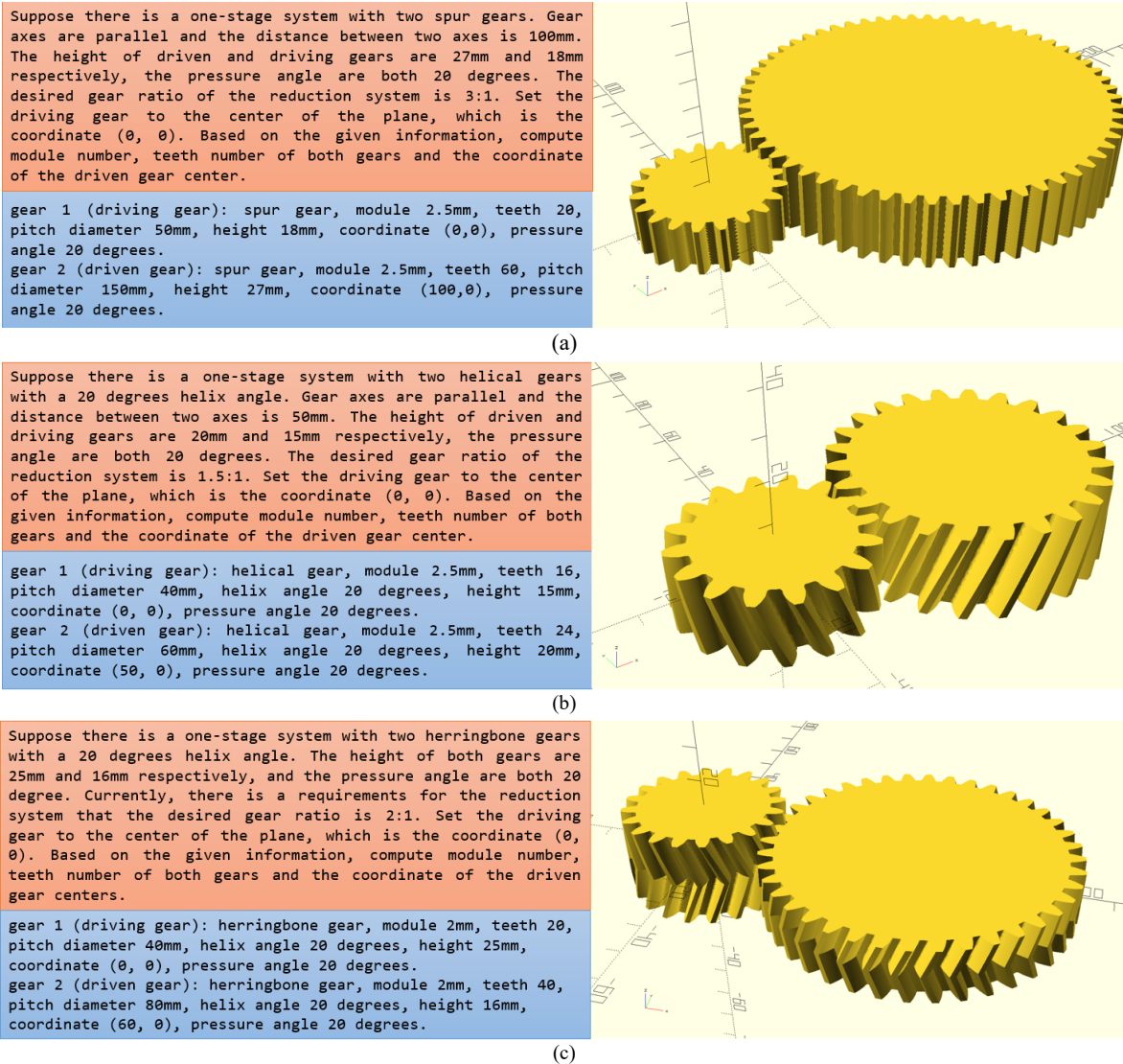


Figure 4. Results in the use case. (a) System of two spur gears (b) System of two helical gears (c) System of two herringbone gears

on the right is the generated CAD model after rendering by the OpenSCAD.

From the result of the use case, GPT-4 can successfully start by reading a textual description of mechanical design, carrying out the computation, and creating desired 3D models with more controllability but without much human intervention. The result can later be edited by users in the OpenSCAD environment without format conversion. The generation process can be treated as a sequence of formal code representation, organization, and translation. Compared to end-to-end generation, intermedium outcomes can be explicitly unrevealed and guided by human beings to avoid error accumulation into the final generation and to refine requirements during the process. Therefore, the result can be more precise with less influence from the hallucination issue and better align with human expectations.

In this use case, GPT-4 acted as an inference engine to gather a series of atomic operations according to the reasoning process to obtain the spatial construction. This concept is referred to as *constructive solid geometry* (CSG). This is a commonly used design diagram in the CAD domain by human engineers. As human designers, we also use a combination of basic operations such as *circle*, *rectangle*, *extrude*, and *remove*

by graphic user interface in commercial CAD software to shape a complex structure. From this perspective, GPT-4 imitates the modeling logic of human beings that is independent of a specific CAD platform or design language. This abstraction can show a promising direction for a more general design automation.

5. Discussion and future works

It is noticed that most critical parameters are given directly in the use case, but in practical situations, critical parameters such as gear height, gear ratio, and gear type should be considered based on desired working conditions of the system such as torque, material properties, and manufacturing issues. In summary, industrial mechanical design is not only modeling using CAD package, but an iterative process that should consider information from different sources to figure out the most satisfying product.

In addition, the complexity of the system that can be generated using the proposed method is still far away from satisfying requirements in practical usage. Except for designing gears (even without other features like hole, step, and fillet), a common one-stage reduction system also contains other

important parts such as shaft, bearing, box, and fasteners. More importantly, the topology relationships among these components in an assembly system require spatial reasoning and trial-and-error verifying. To enable LLMs to generate a system including more various parts with proper topology relationships for assembly will be the next vital step toward a wider application of LLMs in industrial CAD automation.

The complexity and the scalability of components in an assembly system are essential issues for future research as mentioned above. The proposed framework relies on existing functions in a library, and their combinations, to compile a script for 3D shape generation, which will limit its capability to generate non-standard complicated structures. On one hand, enabling LLMs to create new functions according to new requirements by themselves may be an effective way to tackle this problem. On the other hand, other modeling strategies can be considered. For instance, the *boundary representation* approach, short for B-Rep, can represent 3D geometric shapes by describing their boundary elements such as faces, edges, and vertices. It can provide a detailed and precise representation of solid geometry in a hierarchical structure and support more operational sets on geometric features compared to CSG. Therefore, how to extend this framework to enable the LLMs to learn and infer modeling logic for CAD by taking advantage of CSG and B-Rep is an interesting research topic in engaging LLM-powered AI in the CAD realm. Besides, other CAD diagrams such as generative design, topology optimization, and isogeometric analysis can also be integrated into this process for design space exploration.

Finally, the proposed framework is yet to release the full power of LLMs due to the chain-structure implementation and the dependency on hand-crafted prompts in different stages. A promising implementation for enhancement is the LLMs-based multi-agent collaboration framework [20], which allows LLMs to simulate a group of characters for different work. Using this method, the task can be automatically divided and scheduled by the team and implemented with communication among agents. Information in manufacturing and supply chain can then be introduced into CAD design by setting up agents for integrating domain knowledge and real-time information with external toolkits such as web browsers, handbooks, and expert knowledge bases. Therefore, engineers can conduct a more comprehensive and manufacturing-oriented design.

6. Conclusion

This paper proposes a method on how to use large language models to automate requirement analysis, design computation and generate precise CAD models in the conventional design workflow for manufacturing products. Also, a use case for designing a one-stage reduction system with two gears is given to show the effectiveness of the proposed approach. Future directions of this line of research are rich including improving the capability to generate complicated systems with more components for practical application, enabling the integration of other CAD techniques to enhance the complex structure

modelling ability, and an implementation using a multi-agent approach.

Acknowledgments

This research was supported by the Centre for Digital Engineering and Manufacturing at Cranfield University (UK).

Appendix A. Code and results availability

The code and results for this paper are available in: <https://github.com/BlueAsuka/CAD-GPT>.

References

- [1] E. Mansimov, E. Parisotto, J. Lei Ba, and R. Salakhutdinov, "GENERATING IMAGES FROM CAPTIONS WITH ATTENTION".
- [2] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen OpenAI, "Hierarchical Text-Conditional Image Generation with CLIP Latents".
- [3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models", Accessed: Nov. 16, 2023. [Online]. Available: <https://github.com/hojonathanho/diffusion>.
- [4] A. Vaswani et al., "Attention Is All You Need," 2023.
- [5] J. Yu et al., "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation," Jun. 2022, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2206.10789v1>
- [6] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "DREAMFUSION: TEXT-TO-3D USING 2D DIFFUSION".
- [7] C.-H. Lin et al., "Magic3D: High-Resolution Text-to-3D Content Creation," pp. 300–309, Nov. 2022, doi: 10.1109/cvpr52729.2023.00037.
- [8] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," Feb. 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2302.05543v2>
- [9] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Adv Neural Inf Process Syst*, vol. 35, Jan. 2022, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903v6>
- [10] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," Oct. 2022, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629v3>
- [11] Z. Xi et al., "The Rise and Potential of Large Language Model Based Agents: A Survey," 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://github.com/WooodyDy/LLM-Agent-Paper-List>.
- [12] T. Gupta and A. Kembhavi, "Visual Programming: Compositional visual reasoning without training," pp. 14953–14962, Nov. 2022, doi: 10.1109/cvpr52729.2023.01436.
- [13] Y. Shen et al., "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face," Mar. 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2303.17580v3>
- [14] A. Lindenmayer, "Mathematical models for cellular interactions in development I. Filaments with one-sided inputs," *J Theor Biol*, vol. 18, no. 3, pp. 280–299, Mar. 1968, doi: 10.1016/0022-5193(68)90079-9.
- [15] G. Stiny and J. Gips, "Shape Grammars and the Generative Specification of Painting and Sculpture," vol. 71, pp. 125–135, 1972.
- [16] C. Sun, J. Han, W. Deng, X. Wang, Z. Qin, and S. Gould, "3D-GPT: Procedural 3D Modeling with Large Language Models," Oct. 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2310.12945v1>
- [17] OpenAI, "GPT-4 Technical Report," Mar. 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774v3>
- [18] "OpenSCAD - The Programmers Solid 3D CAD Modeller." Accessed: Nov. 16, 2023. [Online]. Available: <https://openscad.org/>
- [19] "Cut your own gears with profile shift - OpenSCAD library by Parkinbot - Thingiverse." Accessed: Nov. 16, 2023. [Online]. Available: <https://www.thingiverse.com/thing/636119>
- [20] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," Aug. 2023, Accessed: Nov. 16, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155v2>